

This is the sample chapter of DevOps with Laravel. You can find the book [here](#).

Fundamentals

The following chapters are available in the basic package.

nginx

If you want to try out the things explained here, the easiest way is to rent a \$6/month droplet on DigitalOcean with everything pre-installed. Just choose the "LEMP" (it stands for Linux, nginx, MySQL, php) image from their marketplace:

Choose an image

OS Marketplace Snapshots Custom images

🔍 LEMP

Explore all Marketplace Solutions ➔

Search results



LEMP 43 on Ubuntu 22.04

[Details](#)

Recommended for you



WordPress 6.1.1 on Ubuntu ...

[Details](#)



cPanel & WHM® RELEASE ...

[Details](#)



Plesk 18.0.49 on Ubuntu 22...

[Details](#)



Docker 20.10.21 on Ubuntu ...

[Details](#)



LAMP on Ubuntu 20.04

[Details](#)



Ghost on Ubuntu 22.04

[Details](#)

You can destroy the droplet at any time so it costs only a few cents to try out things.

I did the same thing for this chapter. This machine comes with a standard nginx installation so the config file is located inside `/etc/nginx/nginx.conf`. That's the file I'm going to edit. The contents of the sample website live inside the `/var/www/html/demo` folder. I use `systemctl` to reload the nginx config:

```
systemctl reload nginx
```

Serving static content

I won't go into too much detail about the basics since nginx is a well-documented piece of software but I try to give a good intro.

First, let's start with serving static content. Let's assume a pretty simple scenario where we have only three files:

- index.html
- style.css
- logo.png

Each of these files lives inside the `/var/www/html/demo` folder. There are no subfolders and no PHP files.

```
# `events` is not important right now..
events {}

http {
    access_log /var/log/nginx/access.log;
    error_log /var/log/nginx/error.log;
    include mime.types;

    server {
        listen 80;
        server_name 138.68.81.14;
        root /var/www/html/demo;
    }
}
```

This is not a production-ready config! It's only for demo purposes.

In an nginx config, there are two important terms: context and directive.

Context is similar to a "scope" in a programming language. `http`, `server`, `location`, and `events` are the contexts in this config. They define a scope where we can configure scope-related things. `http` is global to the whole server. So if we have 10 sites on this server each will log to the `/var/log/nginx/access.log` file which is obviously not good, but it's okay for now.

Another context is `server` which refers to one specific virtual server or site. In this case, the site is `http://138.68.81.14`. Inside the `server` context we can have a `location` context (but we don't have it right now) which refers to specific URLs on this site.

So with contexts, we can describe the "hierarchy" of things:

```

http {
    # Top-level. Applies to every site on your machine.

    server {
        # Virtual server or site-level. Applies to one specific site.

        location {
            # URL-level. Applies to one specific route.
        }
    }
}

```

Inside the contexts we can write **directives**. It's similar to a function invocation or a value assignment in PHP. `listen 80;` is a directive, for example. Now let's see what they do line-by-line.

```

access_log /var/log/nginx/access.log;
error_log /var/log/nginx/error.log;

```

nginx will log every incoming request to the `access.log` file in a format like this: `172.105.93.231 - - [09/Apr/2023:19:57:02 +0000] "GET / HTTP/1.1" 200 4490 "-" "Mozilla/5.0 (Windows NT 6.3; WOW64; Trident/7.0; rv:11.0) like Gecko"`

When something goes wrong it logs it to the `error.log` file. One important thing though, a 404 or 500 response is not considered as error. The `error.log` file contains only nginx-specific errors, for example, if it cannot be started because the config is invalid.

```

include mime.types;

```

Do you remember the good old `include()` function from PHP? The nginx `include` directive does the same thing. It loads another file. `mime.types` is a file located in the same directory as this config file (which is `/etc/nginx`). The content of this file looks like this:

```
types {  
    text/html html htm shtml;  
    text/css css;  
    text/xml xml;  
    # ...  
}
```

As you can see it contains common mime types and file extensions. If we don't include these types nginx will send every response with the `Content-Type: text/plain` header and the browser will not load CSS and javascript properly. With this directive, if I send a request for a CSS file nginx sends a response such as:

▼ Response Headers

Accept-Ranges: bytes

Content-Length: 980

Content-Type: text/css

Date: Sun, 09 Apr 2023 19:44:17 GMT

ETag: "643167ca-3d4"

Last-Modified: Sat, 08 Apr 2023 13:10:34 GMT

Server: nginx/1.18.0 (Ubuntu)

By the way, I didn't write `mimes.type` it comes with nginx by default.

Next up, we have the server-related configs:

```
listen 80;  
server_name 138.68.81.14;
```

This configuration uses HTTP without SSL so it listens on port 80. The `server_name` usually is a domain name, but right I only have an IP address so I use that.

```
root /var/www/html/demo;
```

The `root` directive defines the root folder of the given site. Every filename after this directive will refer to this path. So if you write `index.html` it means `/var/www/html/demo/index.html`

By default, if a request such as this: `GET http://138.68.81.14` comes in nginx will look for an `index.html` inside the root folder. Which, if you remember, exists so it can return to the client.

When the browser parser the HTML and sends a request for the `style.css` the request looks like this: `http://138.68.81.14/style.css` which also exists since it lives inside the root folder.

That's it! This is the bare minimum nginx configuration to serve static content. Once again, it's not production-ready and it's not optimized at all.

nginx doesn't know anything about PHP. If I add an `index.php` to the root folder and try to request it, I get the following response:

```
root@testing:/var/www/html/demo# curl http://127.0.0.1/index.php
<?php
echo datetime();
root@testing:/var/www/html/demo# curl -I http://127.0.0.1/index.php
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Sun, 09 Apr 2023 20:52:51 GMT
Content-Type: text/plain
Content-Length: 24
Last-Modified: Sun, 09 Apr 2023 20:51:14 GMT
Connection: keep-alive
ETag: "64332542-18"
Accept-Ranges: bytes
```

So it returns the content of the file as plain text. Let's fix this!

CGI, FastCGI, php-fpm

As I said, nginx doesn't know how to run and interpret PHP scripts. And it's not only true for PHP. It doesn't know what to do with a Ruby or Perl script either. So we need something that connects the web server with our PHP scripts. This is what CGI does.

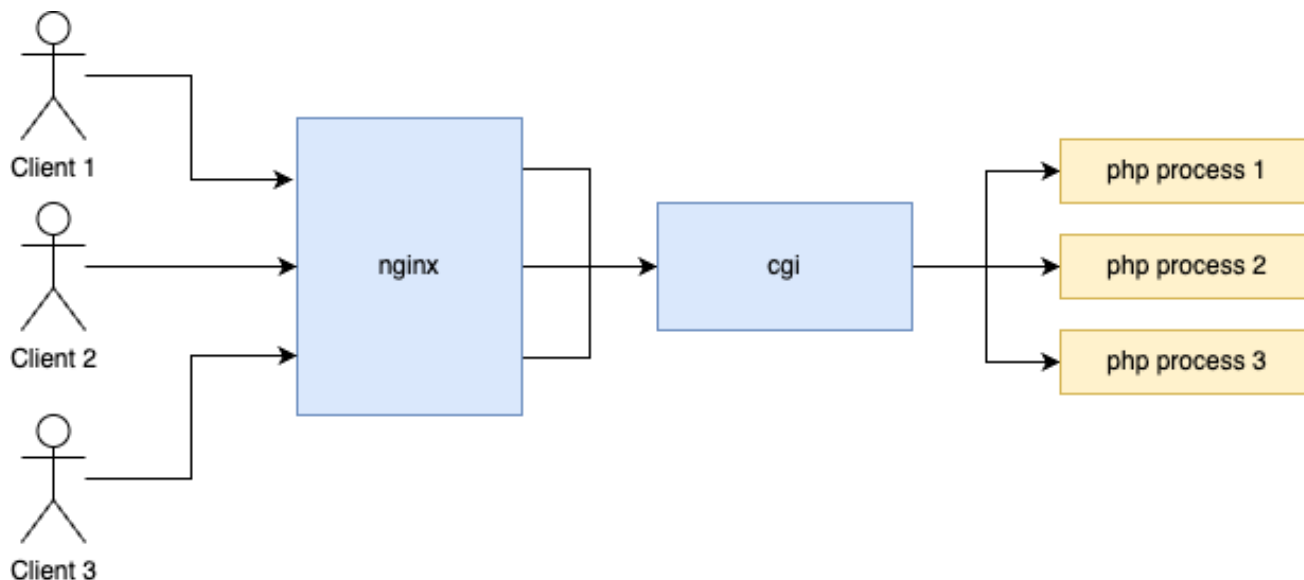
CGI

CGI stands for Common Gateway Interface. As the name suggests, it's not a package or library. No, it's an interface, a protocol. The original specification defines CGI like this:

The Common Gateway Interface (CGI) is a simple interface for running external programs, software or gateways under an information server in a platform-independent manner. - [CGI specification](#)

CGI gives us a unified way to run scripts from web servers to generate dynamic content. It's platform and language-independent so the script can be written in PHP, python, or anything. It can even be a C++ or Delphi program it doesn't need to be a classic "scripting" language. It can be implemented in any language that supports network sockets.

CGI uses a "one-process-per-request" model. It means that when a request comes in to the web browser it creates a new process to execute the php script:

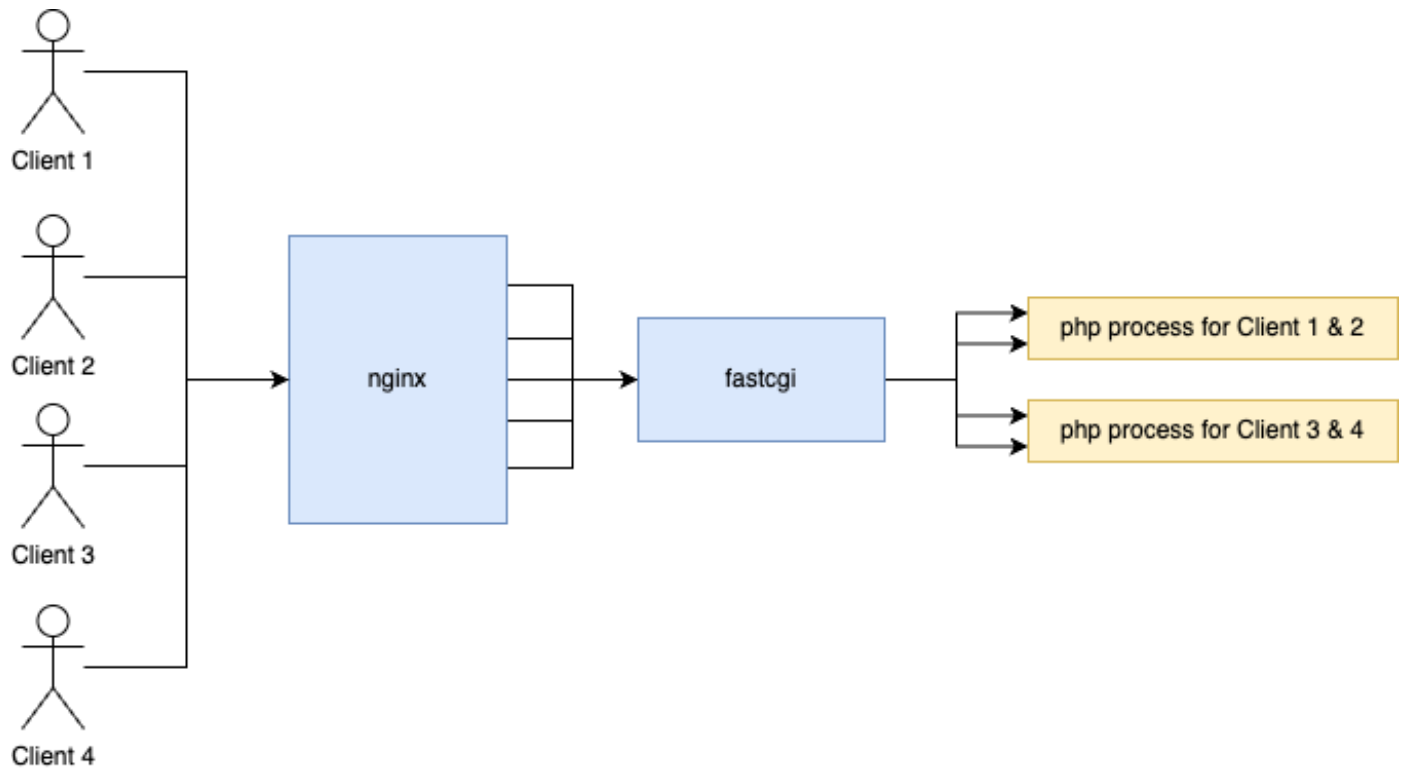


If 1000 requests come in it creates 1000 processes and so on. The main advantage of this model is that it's pretty simple but the disadvantage is that it's pretty resource intensive and hard to scale when there's a high traffic. The cost of creating and destroying processes is quite high. The CPU also needs to switch context pretty often which becomes a costly task when the load is big on the server.

FastCGI

FastCGI is also a protocol. It's built on top of CGI and as its name suggests it's faster. Meaning it can handle more load for us. FastCGI does this by dropping the "one-process-per-request" model. Instead, it has persistent processes which can handle multiple requests by its lifetime so it reduces the CPU overhead of creating/destroying processes and switching between them. FastCGI achieves this by using a technique called multiplexing.

It looks something like that:



FastCGI can be implemented over unix sockets or TCP. We're going to use both of them later.

php-fpm

fpm stands for FastCGI Process Manager. php-fpm is not a protocol or an interface but an actual executable program. A Linux package. This is the component that implements the FastCGI protocol and connects nginx with our Laravel application.

It runs as a separate process on the server and we can instruct nginx to pass every PHP request to php-fpm which will run the Laravel app and return the HTML or JSON response to nginx.

It's a process manager so it's more than just a running program that can accept requests from nginx. It actually has a master process and many worker processes. When nginx sends a request to it the master process accepts it and forwards it to one of the worker processes. The master process is basically a load balancer that distribute the work across the workers. If something goes wrong with one of the workers (for example, exceeding max execution time or memory limit) the master process can kill and restart these processes. It can also scale up and down worker processes as the traffic increases or decreases. php-fpm also helps us avoid memory leaks since it will terminate and respawn worker process after a fixed number of requests.

By the way, this master-worker process architecture is pretty similar to how nginx works (we'll see more about that later).

nginx and PHP

With that knowledge we're now ready to handle PHP requests from nginx! First, let's install php-fpm:

```
apt-get install php-fpm
```

After the installation everything should be ready to go. It should also run as a systemd service which you can check by running these commands:

```
systemctl list-units | grep "php"  
systemctl status php8.0-fpm.service # in my case its php8.0
```

And the output should look like this:

```
root@askmycontent:~# systemctl status php8.0-fpm.service
```

```
● php8.0-fpm.service - The PHP 8.0 FastCGI Process Manager
```

```
Loaded: loaded (/lib/systemd/system/php8.0-fpm.service; enabled; vendor preset: enabled)
```

```
Active: active (running) since Sat 2023-04-22 16:13:13 UTC; 2 days ago
```

```
Docs: man:php-fpm8.0(8)
```

```
Main PID: 779 (php-fpm8.0)
```

```
Dep Status: "Processes active: 0, idle: 2, Requests: 0, slow: 0, Traffic: 0req/sec"
```

```
Tasks: 3 (limit: 2324)
```

```
Memory: 15.1M
```

```
CPU: 17.672s
```

```
CGroup: /system.slice/php8.0-fpm.service
```

```
└─779 "php-fpm: master process (/etc/php/8.0/fpm/php-fpm.conf)"
```

```
└─911 "php-fpm: pool www"
```

```
└─912 "php-fpm: pool www"
```

```
Notice: journal has been rotated since unit was started, output may be incomplete.
```

```
lines 1-15/15 (END)
```

Here's the nginx config that connects requests with php-fpm:

```
user www-data;

events {}

http {
    include mime.types;
    access_log /var/log/nginx/access.log;
    error_log /var/log/nginx/error.log;

    server {
        listen 80;
        server_name 138.68.80.16;

        root /var/www/html/demo;

        index index.php index.html;

        location / {
            try_files $uri $uri/ =404;
        }

        location ~\.php {
            include fastcgi.conf;
            fastcgi_pass unix:/run/php/php-fpm.sock;
        }
    }
}
```

Most of it should be familiar but there are some new directives. Now we have PHP files in our project so it's a good practice to add `index.php` as the index file:

```
index index.php index.html;
```

If it cannot be found nginx will default to an index.html.

Next we have this location scope:

```
location / {  
    try_files $uri $uri/ =404;  
}
```

`try_files` is an exceptional great name because it literally tries to load the given files in order. But what is `$uri` or `=404`

`$uri` is a variable given to us by nginx. It contains the normalized URI from the URL. Here's a few examples:

- `mysite.com` -> `/`
- `mysite.com/index.html` -> `/index.html`
- `mysite.com/posts/3` -> `/posts/3`
- `mysite.com/posts?sort_by=publish_at` -> `/posts`

So if the request contains a specific filename nginx tries to load it. This is what the first part does:

```
try_files $uri
```

If the request is `mysite.com/about.html` then it returns the contents of `about.html`.

What if the request contains a folder name? I know it's not that popular nowadays (or in Laravel) but nginx was published a long time ago. The second parameter of `try_files` makes it possible to request a specific folder:

```
try_files $uri $uri/
```

For example, if the request is `mysite.com/articles` and we need to return the `index.html` from the `articles` folder the `$uri/` makes it possible. This is what happens:

- nginx tries to find a file called `articles` in the root but it's not found
- Because of the `/` in the second parameter `$uri/` it looks for a **folder** named `articles`. Which exists.
- Since in the `index` directive we specified that the index file should be `index.php` or `index.html` it loads the `index.html` under the `articles` folder.

The third parameter is the fallback value. If neither a file nor a folder cannot be found nginx will return a 404 response:

```
try_files $uri $uri/ =404;
```

One important thing. Nginx locations have priorities. So if a request matches two locations it will hit the more specific one. Let's take the current example:

```
location / {}  
location ~\.php {}
```

The first location should essentially match every request since all of them starts with a trailing / However, if a request such as `/phpinfo.php` comes in, the second location will be evaluated since it's more specific to the current request.

So the first location block takes care of static content.

And the second one handles requests for PHP files. Remember, Laravel and user-friendly URLs are not involved just yet. For now, a PHP request means something like `mysite.com/phpinfo.php` with a `.php` in the URL.

To catch these requests we need a location such as this:

```
location ~\.php {}
```

As you can see it's a regex since we want to match any PHP files:

- `~` just means it's a regex and it's case-sensitive (`~*` is used for case-insensitive regexes)
- `\.` is just the escaped version of the `.` symbol

So this will match any PHP file.

Inside the location there's an include:

```
include fastcgi.conf;
```

As we already discussed nginx comes with some predefined configurations that we can use. This file basically defines some basic environment variables to php-fpm. Things like these:

```
fastcgi_param  SCRIPT_FILENAME    $document_root$fastcgi_script_name;  
fastcgi_param  QUERY_STRING       $query_string;  
fastcgi_param  REQUEST_METHOD     $request_method;
```

php-fpm needs informations about the request method, query string, the file that's being executed and so on.

And the last line is where the magic happens:

```
fastcgi_pass unix:/run/php/php-fpm.sock;
```

This instructs nginx to pass the request to php-fpm via a Unix socket. If you remember, FastCGI can be used via Unix sockets or TCP connections. Here we're using the earlier. They provide a way to pass binary data between processes. This is exactly what happens here.

Here's a command to locate the php-fpm socket's location:

```
find / -name *fpm.sock
```

It finds any file named `*fpm.sock` inside the `/` folder (everywhere on the server).

So this is the whole nginx configuration to pass requests to php-fpm:

```
location ~\.php {  
    include fastcgi.conf;  
    fastcgi_pass unix:/run/php/php-fpm.sock;  
}
```

Later, we'll do the same inside docker containers and with Laravel. We'll also talk about how to optimize nginx and php-fpm.

nginx and Vue

When you run `npm run build` it builds the whole frontend into static HTML, CSS, and javascript files that can be served as simple static files. After the browser loads the HTML it sends requests to the API. Since this is the case serving a Vue application doesn't require as much config as serving a PHP API.

However, in the "Serving static content" I showed you a pretty basic config for demo purposes so here's a better one:

```
server {  
    listen 80;  
    server_name 138.68.80.16;  
    root /var/www/html/posts/frontend/dist;  
    index index.html;  
  
    location / {  
        try_files $uri $uri/ /index.html;  
    }  
}
```

As you can see, the `dist` folder is the root. This is where the build command generates its output. The frontend config needs only one location where we try to load:

- The exact file requested. Such as <https://example.com/favicon.ico>
- An index.html file from a folder. Such as <https://example.com/my-folder>
- Finally, we go to the index.html file. Such as <https://example.com/>

This is still not an optimized config but it works pretty fine. We're gonna talk about optimization in a dedicated chapter.

Combined nginx config

The configurations in the last two chapters only work if you have two domains, for example:

- myapp.com
- and api.myapp.com

Which is pretty common. We haven't talked about domains yet, but this would look like this:

```
user www-data;

events {}

http {
    include mime.types;
    access_log /var/log/nginx/access.log;
    error_log /var/log/nginx/error.log;

    # api
    server {
        listen 80;
        server_name api.myapp.com;

        root /var/www/html/demo;

        index index.php index.html;

        location / {
            try_files $uri $uri/ =404;
        }

        location ~\.php {
            include fastcgi.conf;
            fastcgi_pass unix:/run/php/php-fpm.sock;
        }
    }

    server {
        listen 80;
```



```

server_name myapp.ccom;

root /var/www/html/posts/frontend/dist;
index index.html;

location / {
    try_files $uri $uri/ /index.html;
}
}

```

But if you don't want to use a subdomain for the API but a URI:

- myapp.com
- myapp.com/api

Then the config should look like this:

```

server {
    server_name myapp.com www.myapp.com;
    listen 80;
    index index.html index.php;

    location / {
        root /var/www/html/posts/frontend/dist;
        try_files $uri $uri/ /index.html;
        gzip_static on;
    }

    location ~\.php {
        root /var/www/html/posts/api/public;
        try_files $uri =404;
        include /etc/nginx/fastcgi.conf;
        fastcgi_pass unix:/run/php/php8.1-fpm.sock;
        fastcgi_index index.php;
        fastcgi_param PATH_INFO $fastcgi_path_info;
    }

    location /api {
        root /var/www/html/posts/api/public;
    }
}

```

```
    try_files $uri $uri/ /index.php?$query_string;
}
}
```

This config file has only one `server` block since you have only one domain and it has three locations:

- `/` serves the homepage (frontend)
- `/api` serves API requests and `try_files` will send requests to
- `~\.php` which communicates with FPM

As you can see, each location defines its own `root` folder. We're going to use this approach. One domain and the frontend sends request to the backend such as `GET /api/posts`. Later, we're going to use a dedicated reverse proxy to accomplish the same results but in a cleaner way.

Now that we know the basics, let's write a deploy script!

Deploy script

In this chapter, I'm going to write a basic but fully functional bash script that deploys our application. The project is going to live inside the `/var/www/html/posts` folder and the nginx config inside `/etc/nginx`

`deploy.sh`:

```
#!/bin/bash

set -e

MYSQL_PASSWORD=$1

PROJECT_DIR="/var/www/html/posts"
```

The `set -e` command in a bash script enables the shell's `errexit` option, which causes the script to exit immediately if any command exits with a non-zero status code (i.e. if it fails). I highly recommend to start your scripts with this option. Otherwise, if something fails the script will continue. There's also an `x` flag that enables `xtrace` which causes the shell to print each command before it is executed. It can be useful for debugging, however, it's a security risk because the script may print sensitive informations to the output. Which might be stored on the filesystem.

The next line

```
MYSQL_PASSWORD=$1
```

sets a simple variable. It's not an environment variable so it can only be used in the current script. It reads the second parameter of the command which is the database password. A few pages later I'm gonna explain why we need it. So actually, the `deploy.sh` needs to be executed such as this:

```
./deploy.sh hL81sP4t@9%c
```

The last line

```
PROJECT_DIR="/var/www/html/posts"
```

sets another variable. It defines the path of the deployment so we can use absolute paths throughout the script.

The next part looks like this:

```

# make dir if not exists (first deploy)
mkdir -p $PROJECT_DIR

cd $PROJECT_DIR

git config --global --add safe.directory $PROJECT_DIR

# the project has not been cloned yet (first deploy)
if [ ! -d $PROJECT_DIR"/.git" ]; then
    GIT_SSH_COMMAND='ssh -i /home/id_rsa -o IdentitiesOnly=yes' git clone
git@github.com:mmartinjoo/devops-with-laravel-sample.git .
else
    GIT_SSH_COMMAND='ssh -i /home/id_rsa -o IdentitiesOnly=yes' git pull
fi

```

The `mkdir -p` command creates a directory and any necessary parent directories. If the folder already exists it does nothing.

This expression `[ ! -d $PROJECT_DIR"/.git" ]` checks if a `.git` folder exists in the project directory and the `!` operator negates the result. So it becomes true if the `.git` directory does not exist. This means, this is the first deployment and we need to clone project from GitHub:

```

GIT_SSH_COMMAND='ssh -i /home/id_rsa -o IdentitiesOnly=yes' git clone
git@github.com:mmartinjoo/devops-with-laravel-sample.git .

```

Using the `GIT_SSH_COMMAND` variable we can override how git tries to resolve ssh keys. In this case, I specify the exact location of an SSH key in the `/home` directory. Later I'll show you how that file gets there. But for now, the important part is that git needs an SSH key to communicate with GitHub and this key needs to be on the server.

If the `.git` directory already exists the script runs a `git pull` so the project is updated to the latest version.

So far this is what the script looks like:

```

#!/bin/bash

set -e

MYSQL_PASSWORD=$1

```

```
PROJECT_DIR="/var/www/html/posts"

# make dir if not exists (first deploy)
mkdir -p $PROJECT_DIR

cd $PROJECT_DIR

git config --global --add safe.directory $PROJECT_DIR

# the project has not been cloned yet (first deploy)
if [ ! -d $PROJECT_DIR"/.git" ]; then
    GIT_SSH_COMMAND='ssh -i /home/id_rsa -o IdentitiesOnly=yes' git clone
git@github.com:mmartinjoo/devops-with-laravel-sample.git .
else
    GIT_SSH_COMMAND='ssh -i /home/id_rsa -o IdentitiesOnly=yes' git pull
fi
```

We have the source code ready on the server. The next step is to build the frontend:

```
cd $PROJECT_DIR"/frontend"
npm install
npm run build
```

With these three commands, the frontend is built and ready. Every static file can be found inside the `dist` folder. Later, we're going to serve it via nginx.

To get the API ready we need a few steps:

```
composer install --no-interaction --optimize-autoloader --no-dev

# initialize .env if does not exist (first deploy)
if [ ! -f $PROJECT_DIR"/api/.env" ]; then
    cp .env.example .env
    sed -i "/DB_PASSWORD/c\DB_PASSWORD=$MYSQL_PASSWORD"
$PROJECT_DIR"/api/.env"
    sed -i '/QUEUE_CONNECTION/c\QUEUE_CONNECTION=database'
$PROJECT_DIR"/api/.env"
    php artisan key:generate
fi

chown -R www-data:www-data $PROJECT_DIR
```

First, composer packages are being installed. Please notice the `--no-dev` flag. It means that packages in the `require-dev` key will not be installed. They are only required in a development environment.

Next up, we have this line: `if [ ! -f $PROJECT_DIR"/api/.env" ];` It's pretty similar to the previous one but it checks the existence of a single file instead of a directory. If this is the first deploy `.env` will not exist yet, so we copy the example file.

The next line is the reason why we need the database password as an argument:

```
sed -i "/DB_PASSWORD/c\DB_PASSWORD=$MYSQL_PASSWORD" $PROJECT_DIR"/api/.env"
```

This command will write it to the `.env` file so the project can connect to MySQL. Later, I'm gonna show you where the password comes from, but don't worry, we don't need to pass it manually or anything like that. Now let's focus on the `sed` command. To put it simple: it replaces all occurrences of `DB_PASSWORD` to `DB_PASSWORD=foo`. The `-i` flag modifies the file in place, and the `c` command of `sed` replaces the entire line containing `DB_PASSWORD`. By the way, `sed` is a stream editor and is most commonly used for performing string manipulation such as replacing.

We also set the `QUEUE_CONNECTION` to `database`. Later, I'm gonna use Redis but for now, MySQL is perfect.

Remember the

```
user www-data;
```

line in the nginx config? This means that nginx runs everything as `www-data`. For these reasons I make `www-data` the owner of the project directory. The `-R` flag makes the command recursive.

Finally, it's time for Laravel specific commands:

```
php artisan storage:link
php artisan optimize:clear

php artisan down

php artisan migrate --force
php artisan config:cache
php artisan route:cache
php artisan view:cache

php artisan up
```

You probably know most of these commands but let's go through them real quick:

- `php artisan storage:link` creates a symbolic link in the `public` folder that points to the `/storage/app/public` folder. So these files are accessible from the web.
- `php artisan optimize:clear` clears three things: config, route, and view caches. We're deploying a new version of our application. It probably has new configs or routes. This is why we need to clear the old values from the cache.
- `php artisan down` puts the application into maintenance mode so it's not available.
- `php artisan migrate --force` migrates the database without hesitation.
- The `:cache` commands will cache the new values from the new files.
- `php artisan up` ends the maintenance mode.

It's important to cache the configs, routes, and views on every deployment since it makes the performance of the application much better. For example, if you forget to run `config:cache` Laravel will read the `.env` file every time you call something like that: `config('app.my_config')` Assuming that the config file reads `MY_CONFIG` using the `env()` function.

So we're actually done with every application specific code:

```
cd $PROJECT_DIR"/frontend"
npm install
npm run build

cd $PROJECT_DIR"/api"

composer install --no-interaction --optimize-autoloader --no-dev
```

```
# initialize .env if does not exist (first deploy)
if [ ! -f $PROJECT_DIR"/api/.env" ]; then
    cp .env.example .env
    sed -i "/DB_PASSWORD/c\DB_PASSWORD=$MYSQL_PASSWORD"
$PROJECT_DIR"/api/.env"
    sed -i '/QUEUE_CONNECTION/c\QUEUE_CONNECTION=database'
$PROJECT_DIR"/api/.env"
    php artisan key:generate
fi

chown -R www-data:www-data $PROJECT_DIR

php artisan storage:link
php artisan optimize:clear

php artisan down

php artisan migrate --force
php artisan config:cache
php artisan route:cache
php artisan view:cache

php artisan up
```

Next we need to update nginx:

```
sudo cp $PROJECT_DIR"/deployment/config/nginx.conf" /etc/nginx/nginx.conf
# test the config so if it's not valid we don't try to reload it
sudo nginx -t
sudo systemctl reload nginx
```

As you can see, it's quite easy.

It copies the config file stored in the repository into `/etc/nginx/nginx.conf`. This config file contains both the FE and the API just as we discussed in the previous chapter. It's a server with a single application running on it, there are no multiple apps or multi-tenants so we don't need multiple configs and `sites-available` etc. One config file that contains the FE and the API.

`nginx -t` will test the config file and it fails if it's not valid. Since we started the script with `set -e` this command will stop the whole script if something's not right. This means we never try to load invalid config into nginx and it won't crash.

If the test was successful the `systemctl reload nginx` command will reload nginx with the new config. This command is zero-downtime.

Congratulations! You just wrote a fully functional deploy script. We're gonna talk about worker processes in a separate chapter.

Optimization

nginx worker processes and connections

It's finally time to optimize nginx and fpm. Let's start with the low hanging fruits.

```
user www-data;  
worker_processes auto;  
  
events {  
    worker_connections 1024;  
}
```

As you might guessed `worker_processes` controls how many workers can nginx spin up. It's a good practice to set this value equal to the number of CPUs available in your server. The `auto` does this automatically.

`worker_connections` controls how many connection a worker process can open. In Linux, everything is treated as a file, and network sockets are no exception. When a client establishes a connection to a server, a socket file descriptor is created to represent the connection. This file descriptor can be used to read and write data to the connection, just like any other file descriptor.

So this settings controls how many file descriptors a worker process can use. It's important to note that a file descriptor is not a file. It is simply an integer value that represents an open file or resource in the operating system. This can include files on disk, network sockets, pipes, and other types of resources. When a network socket (an IP address and a port) is opened, a file descriptor is created to represent the open socket, but no file is created on disk.

So how can you figure out the right number? Run this on your server:

```
ulimit -n
```

It gives you a number which means how many file descriptor a process can open in Linux. It's usually 1024.

So to summarize: if you have four cores in your CPU and you set `worker_connections` to 1024 (based on `ulimit`) it means that nginx can handle 4096 connections (users) at the same time. If the 4097th user comes in, the connection will be pushed into a queue an it gets served after nginx has some room to breath.

Disclaimer: if you don't have high-traffic spikes and performance problems leave this setting as it is.
`worker_connections === ulimit -n`

However, this means that we limited ourselves to 4k concurrent users even before they hit our API. But wasn't nginx designed to handle 10k+ concurrent connections? Yes, it was. 20 years ago. So we can probably do better.

If you run the following command:

```
ulimit -Hn
```

You get the **real** connection limit for a process. This command gives you the hard limit while `-n` returns the soft limit. In my case, the difference is, let's say, quite dramatic:

```
root@askmycontent:~# ulimit -n
1024
root@askmycontent:~# ulimit -Hn
1048576
root@askmycontent:~#
```

Soft limit means, it's the default, but it can be overwritten. So how can we hack it?

```
user www-data;
worker_processes auto;
worker_rlimit_nofile 2048;

events {
    worker_connections 2048;
}
```

By default, the value of `worker_rlimit_nofile` is set to the system's maximum number of file descriptors, or in other words the value `ulimit -n` returns with. However, we can change it because it's only a "soft" limit.

It's important to note that setting `worker_rlimit_nofile` too high can lead to performance issues and even crashes if the server doesn't have enough resources to handle the load. Only change this if you have real spikes and problems.

fpm processes

php-fpm also comes with a number of configuration that can affect the performance of our servers. These are the most important ones:

- `pm.max_children`: This directive sets the maximum number of fpm child processes that can be started. This is similar to `worker_processes` in nginx.
- `pm.start_servers`: This directive sets the number of fpm child processes that should be started when the fpm service is first started.
- `pm.min_spare_servers`: This directive sets the minimum number of idle fpm child processes that should be kept running to handle incoming requests.
- `pm.max_spare_servers`: This is the maximum number of idle fpm child processes.
- `pm.max_requests`: This directive sets the maximum number of requests that an fpm child process can handle before it is terminated and replaced with a new child process. This is similar to the `--max-jobs` option of the `queue:work` command.

So we can set `max_children` to the number of CPUs, right? Actually, nope.

The number of php-fpm processes is often calculated **based on memory** rather than CPU because PHP processes are typically memory-bound rather than CPU-bound.

When a PHP script is executed, it loads into memory and requires a certain amount of memory to run. The more PHP processes that are running simultaneously, the more memory will be consumed by the server. If too many PHP processes are started, the server may run out of memory and begin to swap, which can lead to performance issues.

TL;DR: if you don't have some obvious performance issue in your code php usually consumes more memory than CPU.

So we need a few information to figure out the correct number for the `max_children` config:

- How much memory does your server have?
- How much memory does a php-fpm process consume on average?
- How much memory does your server need just to stay alive?

Here's a command that will give you the average memory used by fpm processes:

```
ps -ylC php-fpm8.1 --sort:rss
```

- `ps` is a command used to display information about running processes.
- `-y` tells `ps` to display the process ID (PID) and the process's controlling terminal.
- `-l` instructs `ps` to display additional information about the process, including the process's state, the amount of CPU time it has used, and the command that started the process.
- `-C php-fpm8.1` tells `ps` to only display information about processes with the name `php-fpm8.1`.
- `--sort:rss`: will sort the output based on the amount of resident set size (RSS) used by each process.

What the hell is resident set size? It's a memory utilization metric that refers to the amount of physical memory currently being used by a process. It includes the amount of memory that is allocated to the process and **cannot be shared** with other processes. This includes the process's executable code, data, and stack space, as well as any memory-mapped files or shared libraries that the process is using.

It's called "resident" for a reason. It shows the amount of memory that cannot be used by other processes. For example, when you run `memory_get_peak_usage()` in PHP it only returns the memory used by the PHP script. On the other hand, RSS measures the total memory usage of the entire process.

The command will spam your terminal with an output such as this:

```
root@askmycontent:~# ps -ylC php-fpm8.1 --sort:rss
S    UID      PID      PPID    C  PRI   NI   RSS    SZ WCHAN  TTY           TIME CMD
S      0    13789         1    0   80    0 25088 62620 ep_pol ?           00:00:18 php-fpm8.1
S     33    18313    13789    0   80    0 42440 81729 skb_wa ?           00:00:08 php-fpm8.1
S     33    18317    13789    0   80    0 43156 81750 skb_wa ?           00:00:13 php-fpm8.1
S     33    13791    13789    0   80    0 43984 81745 skb_wa ?           00:00:13 php-fpm8.1
root@askmycontent:~#
```

The RSS column shows the memory usage. From 25Mb 43MB in this case. The first line (which has significantly lower memory usage) is usually the master process. We can take that out of the equation and say the average memory used by a php-fpm worker process is 43MB.

However, here are some numbers from a production (older) app:

```
S    1000 13848         1    0   80    0 74408 137609 inet_c ?           00:00:05 php-fpm7.3
S    1000 13841         1    0   80    0 74460 137601 inet_c ?           00:00:06 php-fpm7.3
S    1000 13838         1    0   80    0 74768 137729 inet_c ?           00:00:08 php-fpm7.3
S    1000 13862         1    0   80    0 75000 139088 inet_c ?           00:00:01 php-fpm7.3
S    1000 13846         1    0   80    0 75640 137616 inet_c ?           00:00:04 php-fpm7.3
S    1000 13833         1    0   80    0 76012 137722 inet_c ?           00:00:10 php-fpm7.3
S    1000 13837         1    0   80    0 76916 137746 inet_c ?           00:00:10 php-fpm7.3
S    1000 13806         1    0   80    0 77484 137746 inet_c ?           00:00:16 php-fpm7.3
S    1000 13827         1    0   80    0 80072 139535 inet_c ?           00:00:12 php-fpm7.3
S    1000 13839         1    0   80    0 81156 157003 inet_c ?           00:00:07 php-fpm7.3
S    1000 13840         1    0   80    0 81652 156791 inet_c ?           00:00:07 php-fpm7.3
S    1000 13849         1    0   80    0 81900 156991 inet_c ?           00:00:06 php-fpm7.3
S    1000 13836         1    0   80    0 85164 158663 inet_c ?           00:00:08 php-fpm7.3
```

Yes, these are +130MB numbers.

The next question is how much memory does your server need just to stay alive? This can be determined using `htop`:



As you can see from the load average, right now nothing is happening on this server but it uses ~700MB of RAM. This memory is used by Linux, PHP, MySQL, Redis and all the system components installed on the machine.

So the answers are:

- This server has 2GB of RAM
- It needs 700MB to survive
- On average an fpm process uses 43MB of RAM

This means there is 1.3GB of RAM left to use. So we can spin up $1300/30=30$ fpm processes.

It's a good practice to decrease the available RAM by at least 10% as a kind of "safety margin". So let's calculate with 1.17GB of RAM: $1170/37=28$.

So on this particular server I can probably run 25-30 fpm processes.

Here's how we can determine the other values:

Config	General	This example
pm.max_children	As shown above	28
pm.start_servers	~25% of max_children	7
pm.min_spare_servers	~25% of max_children	7
pm.max_spare_servers	~75% of max_children	21

To be completely honest, I'm not sure how these values calculated but they are the "standard" settings. You can search these configs on the web and you probably run into an article suggestions similar numbers. By the way, there's also a calculator [here](#).

To configure these values we need to edit the `/etc/php/8.1/fpm/pool.d/www.conf` file:

```
pm.max_children = 28
pm.start_servers = 7
pm.min_spare_servers = 7
pm.max_spare_servers = 21
```

I added this file to the example repo inside the `deployment/config/php-fpm` directory. I also changed the deploy script and included these two lines:

```
cp $PROJECT_DIR"/deployment/config/php-fpm/www.conf"  
/etc/php/8.1/fpm/pool.d/www.conf  
systemctl restart php8.1-fpm.service
```

If you remember I used `systemctl reload` for nginx but now I'm using `systemctl restart`. First of all, here's the difference between the two:

- `reload`: reloads the config without stopping and starting the service. It does not cause downtime.
- `restart`: stops and starts the service, effectively restarting it. It terminates all active connections and processes associated with the service and starts it again with a fresh state. It does cause downtime.

Changing the number of children processes requires a full `restart` since fpm needs to kill and spawn processes. This is also true for nginx as well! So if you change the number of worker processes you need to restart it.

Usually when I change an fpm-related config it's almost always the `max_children` and the other process-related configs. However, when I change an nginx config, 99% of the time it's not related to `worker_processes`. So I don't want to cause unnecessary downtime.

In the book we optimize other things such as opcache, gzip, HTTP2, TLS1.3, nginx cache.

Backups and restore

Having a backup of your application's data is crucial for several reasons. Firstly, it provides an extra layer of protection against accidental data loss or corruption. Even the most robust and secure applications can fall victim to unforeseen events like hardware failure, power outages, or even human error. In such cases, having a backup can be the difference between a quick recovery and a catastrophic loss.

Secondly, backups are essential for disaster recovery. In the unfortunate event of a security breach, a backup can help you restore your data to a previous state, reducing the risk of data loss and minimizing the impact on your business.

Here are some important things about having backups:

- Store them on an **external storage** such as S3. Storing the backups on the same server as the application is not a great idea.
- Have **regular** backups. Of course, it depends on the nature of your application and the money you can spend on storage. I'd say having at least one backup per day is the minimum you need to do.
- Keep **at least a week** worth of backups in your storage. It also depends on your app but if you have some bug that causes invalid data, for example, you still have a week of backups to recover from.
- Have a **restore** script that can run at any time. It has to be as seamless as possible. Ideally, the whole process is automated and you only need to run a script or push a button.
- Include the database dump, redis dump, storage folder, and .env file in the backup. Don't include the `vendor` and `node_modules` folders.

In this chapter, we're implement all of these!

AWS S3

One of the best ways to store backups is using AWS S3. Amazon S3 (Simple Storage Service) is a highly scalable, secure, and durable object storage service provided by AWS. It allows us to store and retrieve any amount of data from anywhere on the web, making it a popular choice for cloud-based storage solutions. S3 provides a simple HTTP API that can be used to store and retrieve data from anywhere on the web.

In S3, data is stored in **buckets**, which are essentially containers for objects. Buckets are used to store and organize data, and can be used to host static websites as well. It's important to note that buckets are not folders. It's more similar to a git repository where you can have any number of folders and files (objects). Or the `/` folder in Linux. So it's the root.

Objects are the individual files that are stored in buckets. Objects can be anything from a simple text file to a large video file or database backup (like in our case). Each object is identified by a unique key, which is used to retrieve the object from the bucket.

In my case, the bucket name will be `devops-with-laravel-backups` and we'll store ZIP files in that bucket. We will URLs such as this:


```
https://devops-with-laravel-backups.s3.us-west-2.amazonaws.com/2023-04-30-11-23-05.zip
```

As you can guess from this URL, bucket names must be globally unique.

To create a bucket you need an AWS account. Then login into the console and search for "S3." Then click on "Buckets" in the left navigation. We don't need to set any options, only a bucket name:

The screenshot shows the AWS Management Console interface for creating a new S3 bucket. The top navigation bar includes the AWS logo, a 'Services' menu, a search bar, and a keyboard shortcut '[Option+S]'. The left-hand navigation pane is open to the 'Amazon S3' section, with 'Buckets' selected. The main content area is titled 'Create bucket' and includes a breadcrumb trail: 'Amazon S3 > Buckets > Create bucket'. Below the title is a brief description: 'Buckets are containers for data stored in S3. [Learn more](#)'. The 'General configuration' section contains three main fields: 'Bucket name' with the value 'devops-with-laravel-backups', 'AWS Region' set to 'US East (N. Virginia) us-east-1', and a section for 'Copy settings from existing bucket - optional' with a 'Choose bucket' button. A note states: 'Only the bucket settings in the following configuration are copied.'

Now we have a bucket where we can upload files manually, or using the AWS CLI tool. If you upload a file you can see properties such as these:

Amazon S3 > Buckets > devops-with-laravel-backups > posts/ > 2023-04-30-19-27-14.zip

2023-04-30-19-27-14.zip [Info](#)

[Copy S3 URI](#) [Download](#) [Open](#) [Object actions](#)

[Properties](#) [Permissions](#) [Versions](#)

Object overview

Owner	m4rt1n.j00	S3 URI	s3://devops-with-laravel-backups/posts/2023-04-30-19-27-14.zip
AWS Region	US East (N. Virginia) us-east-1	Amazon Resource Name (ARN)	arn:aws:s3:::devops-with-laravel-backups/posts/2023-04-30-19-27-14.zip
Last modified	April 30, 2023, 21:27:16 (UTC+02:00)	Entity tag (Etag)	234889749ac981d661a8de5ea994c12a
Size	309.3 KB	Object URL	https://devops-with-laravel-backups.s3.amazonaws.com/posts/2023-04-30-19-27-14.zip
Type	zip		
Key	posts/2023-04-30-19-27-14.zip		

There are two URLs:

- S3 URI: <s3://devops-with-laravel-backups/posts/2023-04-30-19-27-14.zip>
- Object URL: <https://devops-with-laravel-backups.s3.amazonaws.com/posts/2023-04-30-19-27-14.zip>

We want to access these backups from the restore bash script where we will use the AWS CLI tool. This tool works with S+ URIs. If you want access files via HTTP you need to use the Object URL.

First, we need to install the AWS CLI tool. You can find the installation guide [here](#), but on Linux it looks like this:

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o
"awscliv2.zip"
unzip awscliv2.zip
./aws/install
```

I added these commands to the `provision_server.sh` script.

After that you need an access token to use authorize. If you're working in a team you should enable the IAM identity center. I'm not going into too much details because it's outside of the scope of this book, and it's well documented [here](#). However, if you're just playing with AWS you can use a root user access key (which is not recommended in a company or production environment!). Just click your username in the upper right corner and click on the "Security credentials" link. There's an "Access keys" section where you can create a new one.

After that you need to configure your CLI. Run this command:

```
aws configure
```

It will ask for the access key.

After that you should be able to run S3 commands. For example, list your buckets:

```
root@staging:~# aws s3 ls
2023-04-30 17:34:31 devops-with-laravel-backups
root@staging:~#
```

Or you can download specific files:

```
(base) → ~ aws s3 cp s3://devops-with-laravel-backups/posts/2023-04-30-19-34-56.zip ./last_backup.zip
download: s3://devops-with-laravel-backups/posts/2023-04-30-19-34-56.zip to ./last_backup.zip
(base) → ~
```

That's the command we need to use when restoring a backup.

To use S3 with Laravel, first we need to set these environment variables:

```
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
AWS_DEFAULT_REGION=us-east-1
AWS_BUCKET=devops-with-laravel-backups
AWS_USE_PATH_STYLE_ENDPOINT=false
```

After that we need to install this package:

```
composer require league/flysystem-aws-s3-v3 "^3.0"
```

And that's it! Everything is ready.

By the way, you don't have to use S3 if you don't want to. To be honest, if it wasn't for the book, I would use DigitalOcean. The UI is 100x times better, it doesn't require a PhD to use, and it's developer-friendly.

A lot of cloud providers offer S3-compatible storage solutions. S3-compatible means it has the same API. DigitalOcean Spaces is a good example.

You can even run your own self-hosted S3 if you'd like to. There's an application called MinIO which is S3-compatible and you can run it on your own server. You can buy a VPS for ~\$6 that has ~25GB of space and MinIO installed on it and you're probably good to go with smaller projects. You can even use SFTP (instead of MinIO) if you'd like to. `spatie/laravel-backup` (which we're gonna use in a minute) supports it.

If you'd like to switch to another S3-compatible storage all you need to do is configure Laravel's filesystem:

```
's3' => [  
    'endpoint' => env('AWS_ENDPOINT', 'https://digital-ocean-spaces-url'),  
],
```

spatie/laravel-backup

There's a pretty useful package by Spatie called `laravel-backup`. As the name suggests, it can create backups from a Laravel app. It's quite straightforward to setup and configure so I'm not gonna go into too much details. They have a great documentation [here](#).

To configure the destination only this option needs to be updated in the `config/backup.php` file:

```
'disks' => [  
    's3',  
],
```

Other than that, we only need to schedule two commands:

```
$schedule->command('backup:clean')->daily()->at('01:00');  
  
$schedule->command('backup:run')->daily()->at('01:30');
```

The `clean` command will delete old backups based on your config while the `run` command will create a new backup.

And that's it! It'll create a ZIP file including your databases and the whole directory of your application.

If you run `backup:run` manually you'll get the following message:

```
(base) → api git:(main) ✗ php artisan backup:run
```

Starting backup...

Dumping database devops-with-laravel...

Determining files to backup...

Zipping 369 files and directories...

Created zip containing 369 files and directories! Size is 71.3 MB

Copying zip to disk named s3...

Successfully copied zip to disk named s3.

Backup completed!

```
(base) → api git:(main) ✗
```

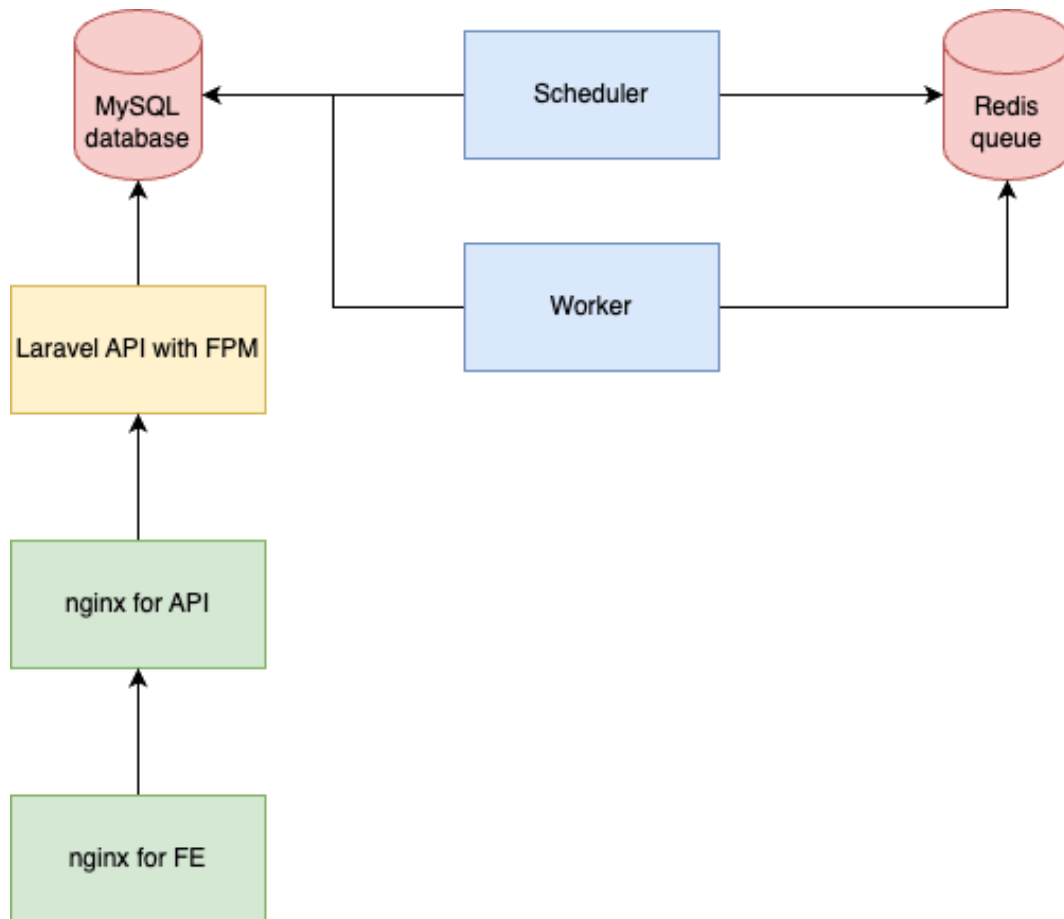
And you should see the file in S3. By the way, laravel-backup will create a folder with the `APP_NAME` inside S3, by default. I left this setting on but you can configure everything in the `config/backup.php` file.

The restore process is discussed only in the book.

Docker

Overview of the application

To dockerize the sample application we need the following architecture:



Each box is a container and they have the following responsibilities:

- nginx for FE serves the static files of the Vue frontend
- nginx for API: this nginx instance accepts requests from the Vue app
- Laravel API with FPM: this is where the application lives. nginx forwards HTTP requests to this container, FPM accepts them, and finally it forwards them to our Laravel app.
- MySQL has a separated container using the official MySQL image
- The scheduler container contains the same Laravel code as "Laravel API with FPM" container. But instead of running PHP FPM, this container will run `php artisan schedule:run` once every 60 seconds. Just like we used crontab in the previous chapters. Scheduler might dispatch queue jobs, and it might also need the database for various reasons.
- The worker container is similar to the scheduler but it runs the `php artisan queue:work` command so it will pick up jobs from the queue. It also contains the source code, since it runs jobs.
- And finally we have a container for Redis. Just like MySQL it uses the official Redis image.

This is the basic architecture of the application. Later, I'm gonna scale containers, use nginx to load balance, use supervisor to scale workers but for now the goal is to dockerize everything and use docker-compose to orchestrate the containers.

Dockerizing a Laravel API

Let's start by writing a Dockerfile to the Laravel API:

```
FROM php:8.1-fpm
```

```
WORKDIR /usr/src
```

The base image is `php:8.1-fpm`. You can use any version you want, but it's important to use the FPM variant of the image. It has PHP and PHP-FPM preinstalled. You can check out the official Dockerfile [here](#).

In Dockerfiles, I used to use `/usr/src` as the root of the project. Basically you can use almost any folder you'd like to. Some other examples I encountered with:

- `/usr/local/src`
- `/application`
- `/laravel`
- `/var/www`
- `/var/www/html`

The next step is to install system packages:

```
RUN apt-get update && apt-get install -y \  
    libpng-dev \  
    libonig-dev \  
    libxml2-dev \  
    libzip-dev \  
    git \  
    curl \  
    zip \  
    unzip \  
    supervisor \  
    default-mysql-client
```

First, we run an `apt-get update` and then installing the following libs:

- `libpng-dev` is needed to deal with PNG files.
- `libonig-dev` is the Oniguruma regular expression library.
- `libxml2-dev` is a widely-used XML parsing and manipulation library written in C.

- `libzip-dev` deals with ZIP files.

These libs are C program and needed by PHP or a particular composer package or Laravel itself. Other than these low-level libs we install standard user-facing programs such as: git, curl, zip, unzip, supervisor.

`default-mysql-client` contains `mysqldump` which is required by `laravel-backup`.

After installing packages, it's recommended to run these commands:

```
RUN apt-get clean && rm -rf /var/lib/apt/lists/*
```

- `apt-get clean` cleans the cache memory of downloaded package files.
- `rm -rf /var/lib/apt/lists/*` removes the lists of available packages and their dependencies. They're automatically regenerated the next time `apt-get update` is run.

The great thing about the official PHP image (compared to starting from Debian and installing PHP manually) is that it has a helper called `docker-php-ext-install`. It can be used to install PHP extensions very easily:

```
RUN docker-php-ext-install pdo_mysql mbstring exif pcntl bcmath gd zip
```

By the way, `docker-php-ext-install` is a [simple shell](#) script included in the official image.

This installs the following extensions:

- `pdo_mysql` is used by Laravel to interact with databases. PDO stand for PHP Data Objects which is another [PHP extension](#) that ships with PHP by default.
- `mbstring` stands for "multibyte string" and provides functions for working with multibyte encodings in PHP. Long time ago one character was one exactly byte. But nowadays, when we have UTF8 characters require more than 1 byte. Hence the name multibyte string.
- `exif` provides functions for reading and manipulating metadata embedded in image files.
- `pcntl` is used for managing processes and signals.
- `bcmath` is a PHP extension that provides arbitrary precision arithmetic functions for working with numbers that are too large or too precise to be represented using the standard floating-point data type.
- `gd` handles in various formats, such as JPEG, PNG, GIF, and BMP. It is required to [create PDFs](#) as well.
- And finally `zip` is pretty self-explanatory.

`docker-php-ext-install` can only install PHP core libraries. Usually, the low-level ones. If you need something else you can use PECL:

```
RUN pecl install redis
```

In this example, I'm going to use Redis as a queue so we need to install it.

In a Dockerfile we also have another, kind of unusual but pretty fast way of installing stuff: copying the binary from another Docker image:

```
COPY --from=composer:2.5.8 /usr/bin/composer /usr/bin/composer
```

There's a `--from` option to the `COPY` command in which we can specify from which Docker image we want to copy files. Composer also has an [official image](#). If you run the image

```
docker run --rm -it composer:2.5.8
```

you can find the `composer` executable file in the `/usr/bin` directory. The `COPY --from=composer:2.5.8 /usr/bin/composer /usr/bin/composer` downloads that file from the composer image and copies it into our own image.

The next line copies the source code from the host machine to the container:

```
COPY . .
```

`.` means the current working directory which is `/usr/src` in the image.

And finally we can copy our PHP and FPM config files. If you remember the project has this structure:

- api
 - Dockerfile
- frontend
- deployment
 - config

The config files are located in `deployment/config` and right now I'm editing the `api/Dockerfile` file. So the `COPY` command looks like this:

```
COPY ../deployment/config/php-fpm/php.ini /usr/local/etc/php/conf.d/php.ini
COPY ../deployment/config/php-fpm/www.conf /usr/local/etc/php-fpm.d/www.conf
```

But this would fail miserably if you run a `docker build` from the `api` folder, such as this: `docker build -t api:0.1 .`

The result is this:

```

=> ERROR [stage-0 11/12] COPY ../deployment/config/php-fpm/php.ini /usr/local/etc/php/conf.d/php.ini
=> ERROR [stage-0 12/12] COPY ../deployment/config/php-fpm/www.conf /usr/local/etc/php-fpm.d/www.conf
-----
> [stage-0 10/12] COPY ./api .:
-----
> [stage-0 11/12] COPY ../deployment/config/php-fpm/php.ini /usr/local/etc/php/conf.d/php.ini:
-----
> [stage-0 12/12] COPY ../deployment/config/php-fpm/www.conf /usr/local/etc/php-fpm.d/www.conf:
-----
Dockerfile:36
-----
34 |
35 |     COPY ../deployment/config/php-fpm/php.ini /usr/local/etc/php/conf.d/php.ini
36 | >>> COPY ../deployment/config/php-fpm/www.conf /usr/local/etc/php-fpm.d/www.conf
37 |
-----
ERROR: failed to solve: failed to compute cache key: failed to calculate checksum of ref 5b072298-d242-4c3e-8780-23314c52f675::kfnsvx16xtfgv5ji7e4bnzi2d: "/deployment/config/
php-fpm/www.conf": not found

```

The reason is that in a Dockerfile you cannot reference folders outside the current working directory. So `../deployment` does not work.

One solution is to move the Dockerfile up:

- api
- frontend
- deployment
- Dockerfile

But this solution is confusing since frontend will also have its own Dockerfile. And of course we might have another services with their own Dockerfiles. So I really want to store this file in the `api` folder.

Another solution is to leave the Dockerfile in the `api` folder but when you build it (or use it in docker-compose) you set the context to the root directory. So you don't build it the `api` but the parent directory:

```

(base) ~/code/devops-with-laravel/docker git:(main) (0.041s)
ls
api          deployment frontend

(base) ~/code/devops-with-laravel/docker git:(main)±26
docker build -t api:0.1 . -f ./api/Dockerfile api/

```

There are three arguments to this command:

- `-t api:0.1` sets the image tag.
- `-f ./api/Dockerfile` sets the Dockerfile we want to build the image from.
- `.` is the context. So it's not the `api` but the current (root) folder.

With a setup like this, we can rewrite the `copy` commands:

```
COPY ./api .
```

```
COPY ./deployment/config/php-fpm/php.ini /usr/local/etc/php/conf.d/php.ini
```

```
COPY ./deployment/config/php-fpm/www.conf /usr/local/etc/php-fpm.d/www.conf
```

`COPY . .` becomes `COPY ./api .` because we are one folder above so the source code is located in `./api`. And the same goes for the deployment folder as well which became `./deployment` instead `../deployment`.

Now the image can be built successfully. However, we don't need to manually build images right now.

The last thing is installing composer packages after the project file have been copied:

```
COPY ./api .
```

```
RUN composer install
```

It's an easy but an important step. Now the image is entirely self-contained, meaning it has:

- Linux
- PHP
- System dependencies
- PHP extensions
- Project files
- Project dependencies

And this is the final result:

```
FROM php:8.1-fpm
```

```
WORKDIR /usr/src
```

```
RUN apt-get update && apt-get install -y \  
    git \  
    curl \  
    libpng-dev \  
    libonig-dev \  
    libxml2-dev \  
    libzip-dev \  

```

```
zip \  
unzip \  
supervisor
```

```
RUN apt-get clean && rm -rf /var/lib/apt/lists/*
```

```
RUN docker-php-ext-install pdo_mysql mbstring exif pcntl bcmath gd zip
```

```
RUN pecl install redis
```

```
COPY --from=composer:2.5.8 /usr/bin/composer /usr/bin/composer
```

```
COPY ./api .
```

```
RUN composer install
```

```
COPY ./deployment/config/php-fpm/php.ini /usr/local/etc/php/conf.d/php.ini
```

```
COPY ./deployment/config/php-fpm/www.conf /usr/local/etc/php-fpm.d/www.conf
```

To summarize:

- It extends the official PHP 8.1 FPM image.
- We install some basic dependencies that are needed by Laravel, the app itself, and the composer packages it uses.
- We install the necessary PHP extensions, Redis, and composer.
- Then we copy files from the host machine into the image.

Before we move on let's build it and run it. The build command is the same as before. Remember, you need to run it from the project root folder (where the `api` folder is located):

```
docker build -t api:0.1 . -f ./api/Dockerfile
```

If you now run the `docker images` command you should see the newly build image:

```
(base) ~/code/devops-with-laravel/docker git:(main) (0.135s)  
docker images  


| REPOSITORY | TAG | IMAGE ID     | CREATED       | SIZE   |
|------------|-----|--------------|---------------|--------|
| api        | 0.1 | fd3df74ded11 | 7 seconds ago | 1.08GB |


```

To run the image execute the following command:

```
docker run -it --rm api:0.1
```

There are two important flags here:

- `-it` will start an interactive shell session inside the container. The `i` flag stands for interactive, which keeps STDIN open, and the `t` flag stands for terminal, which allocates a pseudo-TTY. This means that you can interact with the container's shell as if you were using a local terminal.
- `--rm` will automatically remove the container when it exits. Otherwise Docker would keep the container in a `stopped` status. Removing means more free space on your disk.

After running the command you should see something like this:

```
(base) ~/code/devops-with-laravel/docker git:(main)
docker run -it --rm api:0.1
[02-Jul-2023 12:14:45] NOTICE: fpm is running, pid 1
[02-Jul-2023 12:14:45] NOTICE: ready to handle connections
```

As you can see it started PHP-FPM. But why? Our Dockerfile doesn't do anything except copying files from the host.

If you check out the [php8.1-fpm](#) image you can see it ends with these two commands:

```
EXPOSE 9000
CMD ["php-fpm"]
```

`CMD` is pretty similar to `RUN`. It runs a command, which is `php-fpm` in this case. However, there's a big difference between `RUN` and `CMD`:

- `RUN` is used to execute commands during the **build process** of an image. This can include installing packages, updating the system, or running any other command that needs to be executed to set up the environment for the container. Just as we did.
- `CMD`, on the other hand, is used to define the **default command** that should be executed when a container is started from the image. This can be a shell script, an executable file, or any other command that is required to run the application inside the container.

Since the official PHP image contains a `CMD ["php-fpm"]` command our image will inherit this and run `php-fpm` on startup. Of course we can override it with another `CMD` in our own Dockerfile but we don't need to right now.

There's another command in the PHP image:

```
EXPOSE 9000
```

It means that the container exposes port 9000 to the outside world. We can verify this by running `docker ps`:

```
(base) ~/code/devops-with-laravel/docker git:(main) (0.357s)
docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
31c75e572b07	api:0.1	"docker-php-entrypoi..."	10 minutes ago	Up 10 minutes	9000/tcp	exciting_driscoll

You can see `9000/tcp` is exposed from the container. Right now, we don't need to use it, but later it's going to be very important. This is the port where FPM can be reached by nginx.

`docker ps` also gives us the container ID which we can use to actually go inside the container:

```
docker exec -it 31c75e572b07 bash
```

With `docker exec` you can run command inside your containers. In this example, I run the `bash` command in interactive mode (`-it`) which essentially gives me a terminal inside the container where I can run commands such as `ls -la`:

```
(base) ~/code/devops-with-laravel/docker git:(main)
docker exec -it 31c75e572b07 bash
root@31c75e572b07:/usr/src# ls -la
total 12068
drwxr-xr-x  1 root root    4096 Jul  2 12:11 .
drwxr-xr-x  1 root root    4096 Jun 12 00:00 ..
-rw-r--r--  1 root root   8196 Apr 30 18:37 .DS_Store
-rw-r--r--  1 root root    258 Mar  8 16:57 .editorconfig
-rw-r--r--  1 root root   1217 Jun 30 08:57 .env
-rw-r--r--  1 root root   1087 May  2 17:58 .env.ci
-rw-r--r--  1 root root   1069 Mar  8 16:57 .env.example
-rw-r--r--  1 root root    186 Mar  8 16:57 .gitattributes
-rw-r--r--  1 root root    273 Jun 30 17:56 .gitignore
-rw-r--r--  1 root root   1038 Apr 10 20:20 .phpunit.result.cache
-rw-r--r--  1 root root    796 Jul  2 12:10 Dockerfile
-rw-r--r--  1 root root   4158 Mar  8 16:57 README.md
drwxr-xr-x 12 root root    4096 Apr  7 11:39 app
-rwxr-xr-x  1 root root   1686 Mar  8 16:57 artisan
drwxr-xr-x  3 root root    4096 Mar  8 16:57 bootstrap
-rw-r--r--  1 root root   2161 Apr 30 18:47 composer.json
-rw-r--r--  1 root root  401409 Apr 30 18:47 composer.lock
drwxr-xr-x  2 root root    4096 Jun 30 09:22 config
drwxr-xr-x  5 root root    4096 Apr  6 17:36 database
-rw-r--r--  1 root root   2288 Jul  1 16:22 docker-compose.yml
drwxr-xr-x  2 root root    4096 Jun 30 17:41 frontend
drwxr-xr-x  3 root root    4096 Apr 30 18:44 lang
-rw-r--r--  1 root root    226 Mar  8 16:57 package.json
-rw-r--r--  1 root root 1179328 Jun 13 23:37 php.tar.xz
-rw-r--r--  1 root root    833 Jun 13 23:37 php.tar.xz.asc
-rw-r--r--  1 root root    111 Apr  7 11:45 phpstan.neon
-rw-r--r--  1 root root   1079 Apr 10 17:43 phpunit.xml
drwxr-xr-x  3 root root    4096 Jul  2 12:11 public
drwxr-xr-x  5 root root    4096 Apr  6 17:36 resources
drwxr-xr-x  2 root root    4096 Apr  6 20:09 routes
drwxrwxrwx  8 root root    4096 Jun 30 17:55 storage
drwxr-xr-x  4 root root    4096 Apr  6 17:36 tests
drwxr-xr-x 55 root root    4096 Jun 30 05:19 vendor
-rw-r--r--  1 root root    263 Mar  8 16:57 vite.config.js
-rwxr-xr-x  1 root root   5227 Jun 30 04:58 wait-for-it.sh
root@31c75e572b07:/usr/src#
```

As you can see, we are in the `/usr/src` directory and the project's files are copied successfully. Here you can run commands such as `composer install` or `php artisan tinker`.

There's only one problem, though:

```
root@31c75e572b07:/usr/src# whoami
root
root@31c75e572b07:/usr/src#
```

We are running the container as root. Which is not a really good idea. It can pose a security risk. When a container runs as root, it has root-level privileges on the host system, which means that it can potentially access and modify any file or process on the host system. This can lead to accidental or intentional damage to the host system.

It can also cause annoying bugs. For example, if the storage directory doesn't have 777 permissions and is owned by root, Laravel is unable to write into the log files, etc.

By default, Docker containers run as the root user, but it is recommended to create a new user inside the container. So let's do that!

```
RUN useradd -G www-data,root -u 1000 -d /home/martin martin
```

This is how you create a new user in Linux:

- `-G www-data,root` adds the user to two groups, `www-data` and `root`. If you search for `www-data` in the `fpm` Dockerfile you can see that `FPM` is running as `www-data` and this user is in the `www-data` group. So it's important that our own user is also part of that group.
- `-u 1000` specifies the user ID for the new user. UID is really just an integer number, 1000 is not special at all but it is commonly used as the default UID for the first non-root user created on a system. The UID 0 is reserved for root.
- `-d /home/martin` sets the home directory.
- `martin` is the name of my user.

After that we need to run this:

```
RUN mkdir -p /home/martin/.composer && \  
    chown -R martin:martin /home/martin && \  
    chown -R martin:martin /usr/src
```

It creates a folder for composer and then sets the ownerships of the home and `/usr/src` directories to the new user.

And at the end of the Dockerfile we need to specify that the container should run as `martin`:

```
USER martin
```

These commands need to run before we copy files into the container, so the new Dockerfile looks like this:

```
FROM php:8.1-fpm
```

```
WORKDIR /usr/src
```

```
RUN apt-get update && apt-get install -y \  
    git \  
    curl \  
    libpng-dev \  
    libonig-dev \  
    libxml2-dev \  
    libzip-dev \  
    zip \  
    unzip \  
    supervisor
```

```
RUN apt-get clean && rm -rf /var/lib/apt/lists/*
```

```
RUN docker-php-ext-install pdo_mysql mbstring exif pcntl bcmath gd zip
```

```
RUN pecl install redis
```

```
COPY --from=composer:2.5.8 /usr/bin/composer /usr/bin/composer
```

```
RUN useradd -G www-data,root -u 1000 -d /home/martin martin
```

```
RUN mkdir -p /home/martin/.composer && \  
    chown -R martin:martin /home/martin && \  
    chown -R martin:martin /usr/src
```

```
COPY ./api .
```

```
RUN composer install
```

```
COPY ./deployment/config/php-fpm/php.ini /usr/local/etc/php/conf.d/php.ini
```

```
COPY ./deployment/config/php-fpm/www.conf /usr/local/etc/php-fpm.d/www.conf
```

```
USER martin
```

Let's build a new image and then check the output of `whoami` again:

```
(base) ~/code/devops-with-laravel/docker git:(main) (0.212s)
docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS          NAMES
7d31984002b3   api:0.1    "docker-php-entrypoi..." 7 seconds ago  Up 5 seconds  9000/tcp       elated_hamilton

(base) ~/code/devops-with-laravel/docker git:(main)
docker exec -it 7d31984002b3 bash
martin@7d31984002b3:/usr/src$ whoami
martin
martin@7d31984002b3:/usr/src$
```

Now the container's running as `martin`.

This should be perfectly fine, however, we can make the whole thing dynamic. So we don't need to hardcore the user `martin` in the Dockerfile but rather we can use build arguments. To do that, we need to add two lines at the beginning of the file:

```
FROM php:8.1-fpm
```

```
WORKDIR /usr/src
```

```
ARG user
```

```
ARG uid
```

These are called build arguments but they really just variables. We can pass anything when building the image (or later, running the image from docker-compose) and we can reference it in the Dockerfile:

```
RUN useradd -G www-data,root -u $uid -d /home/$user $user
```

```
RUN mkdir -p /home/$user/.composer && \
  chown -R $user:$user /home/$user && \
  chown -R $user:$user /usr/src
```

```
COPY ./api .
```

```
RUN composer install
```

```
COPY ./deployment/config/php-fpm/php.ini /usr/local/etc/php/conf.d/php.ini
```

```
COPY ./deployment/config/php-fpm/www.conf /usr/local/etc/php-fpm.d/www.conf
```

USER \$user

If you now run the same `docker build -t api:0.1 . -f ./api/Dockerfile` command it fails with this error message:

The command `useradd` failed with an invalid user ID since we didn't pass it the `docker build`. So this is the correct command:

```
docker build -t api:0.1 -f ./api/Dockerfile --build-arg user=joe --build-arg uid=1000 .
```

After building and running the image, you can see it's running as `joe`:

```
(base) ~/code/devops-with-laravel/docker git:(main) (0.297s)
docker ps

| CONTAINER ID | IMAGE   | COMMAND                  | CREATED       | STATUS       | PORTS    | NAMES                |
|--------------|---------|--------------------------|---------------|--------------|----------|----------------------|
| b9930207b88b | api:0.1 | "docker-php-entrypoi..." | 3 seconds ago | Up 2 seconds | 9000/tcp | frosty_chandrasekhar |

(base) ~/code/devops-with-laravel/docker git:(main)
docker exec -it b9930207b88b bash
joe@b9930207b88b:/usr/src$ whoami
joe
joe@b9930207b88b:/usr/src$
```

So this is the final Dockerfile for the Laravel API:

```
FROM php:8.1-fpm
```

```
WORKDIR /usr/src
```

```
ARG user
```

```
ARG uid
```

```
RUN apt-get update && apt-get install -y \  
    git \  
    curl \  
    libpng-dev \  
    libonig-dev \  
    libxml2-dev \  
    libzip-dev \  
    zip \  
    unzip \  
    supervisor
```

```
RUN apt-get clean && rm -rf /var/lib/apt/lists/*
```

```
RUN docker-php-ext-install pdo_mysql mbstring exif pcntl bcmath gd zip
```

```
RUN pecl install redis
```

```
COPY --from=composer:2.5.8 /usr/bin/composer /usr/bin/composer
```

```
RUN useradd -G www-data,root -u $uid -d /home/$user $user
```

```
RUN mkdir -p /home/$user/.composer && \  
    chown -R $user:$user /home/$user && \  
    chown -R $user:$user /usr/src
```

```
COPY ./api .
```

```
RUN composer install
```

```
COPY ./deployment/config/php-fpm/php.ini /usr/local/etc/php/conf.d/php.ini
```

```
COPY ./deployment/config/php-fpm/www.conf /usr/local/etc/php-fpm.d/www.conf
```

USER \$user

After all, it wasn't that complicated.

In the book, we talk about Docker a lot more.

docker-compose

docker-compose is an amazing container orchestration tool that you can use in production and on your local machine as well. Container orchestration is really just a fancy word to say: you can run and manage multiple containers with it.

Right now, if you'd like to run the project you'd need to run all the containers one-by-one which is not quite convenient. docker-compose solves that problem by having a simple `docker-compose.yml` configuration file. The file is located in the root folder at the same level as the `api` or the `frontend` folders:

- `api`
- `frontend`
- `deployment`
- `docker-compose.yml`

Right now, the goal is to create a compose file that works for local development. Production comes later (however, it's not that different).

Frontend

Let's start with the frontend:

```
version: "3.8"
services:
  frontend:
    build:
      context: .
      dockerfile: ./frontend/Dockerfile
      target: dev
    ports:
      - "3000:8080"
```

That's the most basic configuration:

- `version` defined the compose file version you want to use. 3.8 is the newest right now.
- `services` defines a set of services your project consists of. In our case, we're gonna have services such as `frontend`, `api`, `scheduler`, `worker`, etc.
- `frontend` is one of the services.
- `ports` is an important part. If you remember the Dockerfile has an `EXPOSE 8080` instruction. This means that the container exposes port 8080. In the docker-compose file can bind port from the container to the host machine. So `3000:8080` means that `localhost:3000` should be mapped to `container:8080` where container is a made up name referring to the frontend container.

- `build`: just like with the `docker build` command docker-compose will build image when we run it.

Remember when we used the `docker build` command? It looked something like this:

```
docker build -t frontend:0.1 -f ./frontend/Dockerfile .
```

`-f ./frontend/Dockerfile` is the same as `dockerfile: ./frontend/Dockerfile` and the `.` is the equivalent of `context: .`

However, the full `build` command looked like this:

```
docker build -t frontend:0.1 -f ./frontend/Dockerfile --target=dev .
```

Right now, docker-compose doesn't provide the target so let's do these now:

```
version: "3.8"
services:
  frontend:
    build:
      context: .
      dockerfile: ./frontend/Dockerfile
      target: dev
    ports:
      - "3000:8080"
```

With `target` we can target specific build stages. In this case, I'd like to run a dev container.

The purpose of the frontend dev container is to run a development server with hot reload. So whenever you change a file Vue should restart the server and reload the page. To do this we need some real-time connection between the files on your local machine and the files inside the container. We copy files in the Dockerfile but that's not real time, of course. That happens at build time and that's it.

To solve this problem we can use docker volumes:


```
version: "3.8"
services:
  frontend:
    build:
      context: .
      dockerfile: ./frontend/Dockerfile
      target: dev
    ports:
      - "3000:8080"
    volumes:
      - ./frontend:/usr/src
```

Basically, we can bind a folder from the host machine to a folder in the container. This means whenever you change a file in the `frontend` folder (or its subfolders) it gets copied into the container.

And of course we can provide environment variables to the container as well:

```
version: "3.8"
services:
  frontend:
    build:
      context: .
      dockerfile: ./frontend/Dockerfile
      target: dev
    ports:
      - "3000:8080"
    volumes:
      - ./frontend:/usr/src
    environment:
      - NODE_ENV=local
```

That's all the config we need for the frontend. Now let's take care about the api.

API

This is what the `api` services looks like in `docker-compose.yml`:

```
api:
  build:
    args:
      user: martin
      uid: 1000
    context: .
    dockerfile: ./api/Dockerfile
    target: api
  restart: unless-stopped
  volumes:
    - ./api/app:/usr/src/app
    - ./api/config:/usr/src/config
    - ./api/database:/usr/src/database
    - ./api/routes:/usr/src/routes
    - ./api/storage:/usr/src/storage
    - ./api/tests:/usr/src/tests
    - ./api/.env:/usr/src/.env
```

Only one new keyword which is `restart`. As I said earlier, docker-composer compose is a container orchestrator so it can start, restart and manage containers. `restart` makes it possible to automatically restart containers if they stop. It has the following values:

- `no`: Containers should never be automatically restarted, no matter what happens.
- `always`: Containers should always be automatically restarted, no matter what happens.
- `on-failure`: Containers should be automatically restarted if they fail, but not if they are stopped manually.
- `unless-stopped`: Containers should always be automatically restarted, unless they are stopped manually.

`always` is a bit tricky because it'll restart containers that you manually stopped on purpose.

The difference between `on-failure` and `unless-stopped` is this: `on-failure` only restarts containers if they fail so if they exit with a non-0 status code. `unless-stopped` will restart the container even if it stopped with 0.

In the case of the API it's not a big difference since php-fpm is a long-running process and it won't stop with 0. However, when we run a scheduler container it'll stop immediately with status of 0 (because it runs `schedule:work` and then it stops). So in this case, `unless-stopped` is best option. In general, most of the time I use `unless-stopped`.

The next weird thing is this:

```
volumes:
  - ./api/app:/usr/src/app
  - ./api/config:/usr/src/config
  - ./api/database:/usr/src/database
  - ./api/routes:/usr/src/routes
  - ./api/storage:/usr/src/storage
  - ./api/tests:/usr/src/tests
  - ./api/.env:/usr/src/.env
```

Why `./api:/usr/src` isn't enough? If you check out these folders there's an important one that is missing. It's `vendor`.

So in the Dockerfile we installed composer packages, right? If we mount `./api` to `/usr/src` Docker creates a shared folder on your machine. It builds the image, so it has a `vendor` folder. Then it copies the files from `./api` to the shared folder. At this point, there's no `vendor` folder on your local machine. You just pulled the repo and now you want to start the project.

It's important to note that existing files or directories in the container at the specified mount point will not be copied to the host machine. Only files that are created or modified after the volume is mounted will be shared between the host and container. So when the project is running Docker will delete the `vendor` folder from the container! To be honest, I'm not sure why is that but you can try it. Just delete the `vendor` folder on your host, change the volume to `./api:/usr/src` and run `docker-compose up`.

So there are two solutions to that problem:

- The one I just showed you: mounting every folder but `vendor`
- Running `composer-install` in `docker-composer.yml`

The second one is also a valid option, and it looks like this:

```
api:
  build: ...
  command: sh -c "composer install && php-fpm"
  volumes:
    - ./api:/usr/src
```

We run `composer install` every time the container is started. I, personally don't like this option because it's not part of how the containers should be orchestrated, therefore it shouldn't be in `docker-compose.yml`. And it's not just about separation of concern but this solution causes real problems when it comes to scaling (later on that).

That's the reason I use multiple volumes for the different folders.

If you wonder why only those folders and files included: you only need volumes for the files you're editing while writing code. You certainly won't edit `artisan` or the contents of the `bootstrap` folder so you don't need to mount those.

However, this solution has a drawback: you won't have a `vendor` folder on your host machine. Meaning, there's no autocompletion in your IDE. We can solve this problem by copying the folder from the container:

```
#!/bin/sh

ID=$(docker ps --filter "name=api" --format "{{.ID}}")

docker cp $ID:/usr/src/vendor ./api
```

You can find this script as `copy-vendor.sh`. You only need to run it when you initialize a project or when you install new packages. It's a good trade-off in my opinion.

This config doesn't work just yet, because we need MySQL and Redis.

The chapter continues in the book.

Docker Swarm

The following chapters are available in plus package as a separate book.

The project files are located in the `5-swarm` folder.

I know Docker Swarm is not the sexiest thing in the world, but here's my offer:

If you already experienced with docker-compose, you can go from a single machine setup to a highly available 100-server cluster in ~24 hours by learning just ~10 new commands.

Also:

- Deploys are going to be near zero-downtime.
- **You can use the same `docker-compose.yml` as before.**
- Adding or removing servers are no-brainers.
- Scaling out services requires one line of code.
- Rolling back to previous versions is easy.
- The number of options related to updating or rolling back services are huge.
- Your local environment remains the same.
- It also works on one machine.
- Switching from Swarm to Kubernetes is not that hard. The principles are almost the same.
- It's a "native" tool developed by the Docker team.

Now that you are excited, let's talk about the downsides of scaled applications.

State

State is the number one enemy of distributed applications. By distributed, I mean, running on multiple servers. Imagine that you have an API running on two different servers. Users can upload and download files. You are using Laravel's default local storage.

- User A uploads `1.png` to `/storage/app/public/1.png` on Server 1.
- User B wants to download the image but his request gets served by Server 2. But there's no `/storage/app/public/1.png` on Server 2 because User A uploaded it onto Server 1.

So state means when you store something on the current server's:

- Filesystem
- Or memory

Here are some other examples of state:

- Databases such as MySQL. MySQL not just uses state, it is the state itself. So you cannot just run a MySQL container on a random node or in a replicated way. Being replicated means that, for example, 4 containers are running at the same time on multiple hosts. This is what we want to do with **stateless** services but not with a database.
- Redis also means state. The only difference is that it uses memory (but it also persists data on the SSD).
- Local storage just as we discussed a earlier.
- File-based sessions (`SESSION_DRIVER=file`).
- File-based cache (`CACHE_DRIVER=file`).
- `.env` file (kind of...)

When I deployed my first distributed application it took me 4-6 hours of debugging to realize these facts.

All of these problems can be solved relatively easy, so don't worry, we're going to look into different solutions.

Creating a cluster

I recommend you to rent a few \$6 droplets to create a playground cluster with me. Choose the Docker image and name them node1, node2, etc.

To run a cluster you need to open the following ports **on all nodes**:

- 2377
- 7949
- 4789

They are used by Docker for communication. On DigitalOcean droplets you can run this command:

```
ufw allow 2377 && ufw allow 7949 && ufw allow 4789
```

You also need to login into your Docker registry **on every node**. For me, it's Docker Hub so the command is simply:

```
docker login -u <username> -p <token>
```

And here's the command to create a cluster:

```
docker swarm init
```

You're basically done! Now you have a 1-node cluster. This node is the **leader**. If you run this command it gives a token. This token can be used to join the cluster as a **worker** node.

Create another droplet and run this command:

```
docker swarm join --token <your-token> 1.2.3.4:2377
```

1.2.3.4 is the leader's IP address and 2377 is one of the ports you opened before.

Scaling services

API and nginx

And here comes the fun part. Let's scale out the api to 6 replicas:

```
api:
  image: martinjoo/posts-api:${IMAGE_TAG}
  command: sh -c "/usr/src/wait-for-it.sh mysql:3306 -t 60 && /usr/src/wait-for-it.sh redis:6379 -t 60 && php-fpm"
  deploy:
    replicas: 6
  ...
```

Re-deploy the stack:

```
export $(cat .env)
docker stack deploy -c docker-compose.prod.yml posts
```

It's all done.

If you now list the service by running `docker services ls` you should see 6/6 replicas:

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
js7mpg4f5gia	monitoring_visualizer	replicated	1/1	dockersamples/visualizer:stable	*:8080->8080/tcp
87vcsqvb5vp0	posts_api	replicated	6/6	martinjoo/posts-api:a5b15afc99f881b6475fbd30de800ea9a29a1881	
oum3qb62u65w	posts_frontend	replicated	1/1	martinjoo/posts-frontend:a5b15afc99f881b6475fbd30de800ea9a29a1881	*:80->80/tcp

You can list each of these tasks by running `docker service ps posts_api`:

ID	NAME	IMAGE	image: martinjoo/posts-api:\${IMAGE_TAG}	node	DESIRED STATE	CURRENT STATE	ERROR	PORTS
TS			command: sh -c "/usr/src/wait-for-it.sh mysql:3306 -t 60 && /usr/src/wait-for-it.sh redis:6379 -t 60 && php-fpm"					
i87ty04qz7o5	posts_api.1	martinjoo/posts-api:a5b15afc99f881b6475fbd30de800ea9a29a1881		node2	Running	Running 2 hours ago		
xf805ecy35nx	_ posts_api.1	martinjoo/posts-api:513997698d3eac459a14acbad576c864a1b3babf		node2	Shutdown	Shutdown 2 hours ago		
ihgjt7aj6uk1is	posts_api.2	martinjoo/posts-api:a5b15afc99f881b6475fbd30de800ea9a29a1881		node4	Running	Running 2 hours ago		
zfdgmzc86vrk	_ posts_api.2	martinjoo/posts-api:513997698d3eac459a14acbad576c864a1b3babf		node4	Shutdown	Shutdown 2 hours ago		
pjnojhdf5fes	posts_api.3	martinjoo/posts-api:a5b15afc99f881b6475fbd30de800ea9a29a1881		node3	Running	Running 2 hours ago		
ums7x14rt8sy	_ posts_api.3	martinjoo/posts-api:513997698d3eac459a14acbad576c864a1b3babf		node3	Shutdown	Shutdown 2 hours ago		
hda8tf3cexpu	posts_api.4	martinjoo/posts-api:a5b15afc99f881b6475fbd30de800ea9a29a1881		node1	Running	Running 2 hours ago		
zkdzkdzlx7k6	_ posts_api.4	martinjoo/posts-api:513997698d3eac459a14acbad576c864a1b3babf		node1	Shutdown	Shutdown 2 hours ago		
zek0sn4d32a3	posts_api.5	martinjoo/posts-api:a5b15afc99f881b6475fbd30de800ea9a29a1881		node2	Running	Running 2 hours ago		
el565h6gm6q	_ posts_api.5	martinjoo/posts-api:513997698d3eac459a14acbad576c864a1b3babf		node2	Shutdown	Shutdown 2 hours ago		
5kx0sa59a310	posts_api.6	martinjoo/posts-api:a5b15afc99f881b6475fbd30de800ea9a29a1881		node4	Running	Running 2 hours ago		
cenmxc6kcbng	_ posts_api.6	martinjoo/posts-api:513997698d3eac459a14acbad576c864a1b3babf		node4	Shutdown	Shutdown 2 hours ago		

You can see that each node has some replicas of the `api` service. in this picture, you can see two kinds of desired state:

- Running
- Shutdown

It's because I was already running a replicated stack, and then I re-deployed it. When you run `docker stack deploy` Swarm will shutdown the currently running tasks and start new ones.

Scaling nginx vs the API

There's a frequently asked question when it comes to scaling an API: should I scale the nginx service that acts like a reverse proxy or the API itself or both?

To answer that, let's think about what nginx does in our application:

```
location ~\.php {  
    try_files $uri =404;  
    include /etc/nginx/fastcgi_params;  
    fastcgi_pass api:9000;  
    fastcgi_index index.php;  
    fastcgi_param PATH_INFO $fastcgi_path_info;  
    fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;  
}
```

It acts like a reverse proxy. It literally just receives requests and then forwards them right to the API. It's not a performance-heavy task.

If you just run in 1 replica on a server with 2 CPUs it means that it will spin up 2 worker processes with 1024 connections each. So it can handle a maximum of 2048 **concurrent** requests. Of course, it's not a guarantee, it's a maximum number. But it's quite a big number. Now, I guarantee you that the bottleneck will be PHP and MySQL not nginx.

In my opinion, you can scale your nginx but it won't bring you that many benefits. Meanwhile scaling the API is much more useful.

How many replicas?

Usually, the easiest way to decide on the number of replicas is to (kind of) match the number of CPUs in your server/cluster. My cluster has 4 nodes with 2 CPUs each. That's 8 CPUs. So in theory I can scale out the API to 8 replicas and Swarm will probably place 2 on each node. Which is great. But what about other services? Let's say node1 runs two API containers. But it also runs a worker. And nginx. And also the scheduler.

It's easy to see the problem. We have 8 CPUs so we just scale this way:

Container	Number of replicas
API	8
nginx	8
worker	8

(highlighting only the most important containers)

It's **important** to note that in Swarm I don't use supervisor. So a worker container just runs a `queue:work` process (details later).

This chapter continues in the book.

Kubernetes

The following chapters are available in premium package as a separate book.

The project files are located in the `7-kubernetes` folder.

Introduction

Kubernetes is probably the most frequently used orchestrator platform. It is popular for a reason. First of all, it has autoscaling. It can scale not just containers but also nodes. You can define rules, such as: "I want the API to run at least 4 replicas but if the traffic is high let's scale up to a maximum of 8" and you can do the same with your actual servers. So it's quite powerful.

It has every feature we discussed in the Docker Swarm chapter. It can apply resource limits and requests to containers as well which is a pretty nice optimization feature. It has great auto-healing properties.

So when you use Kubernetes it almost feels like you have a new Ops guy on your project. And in fact, it's not that hard to learn.

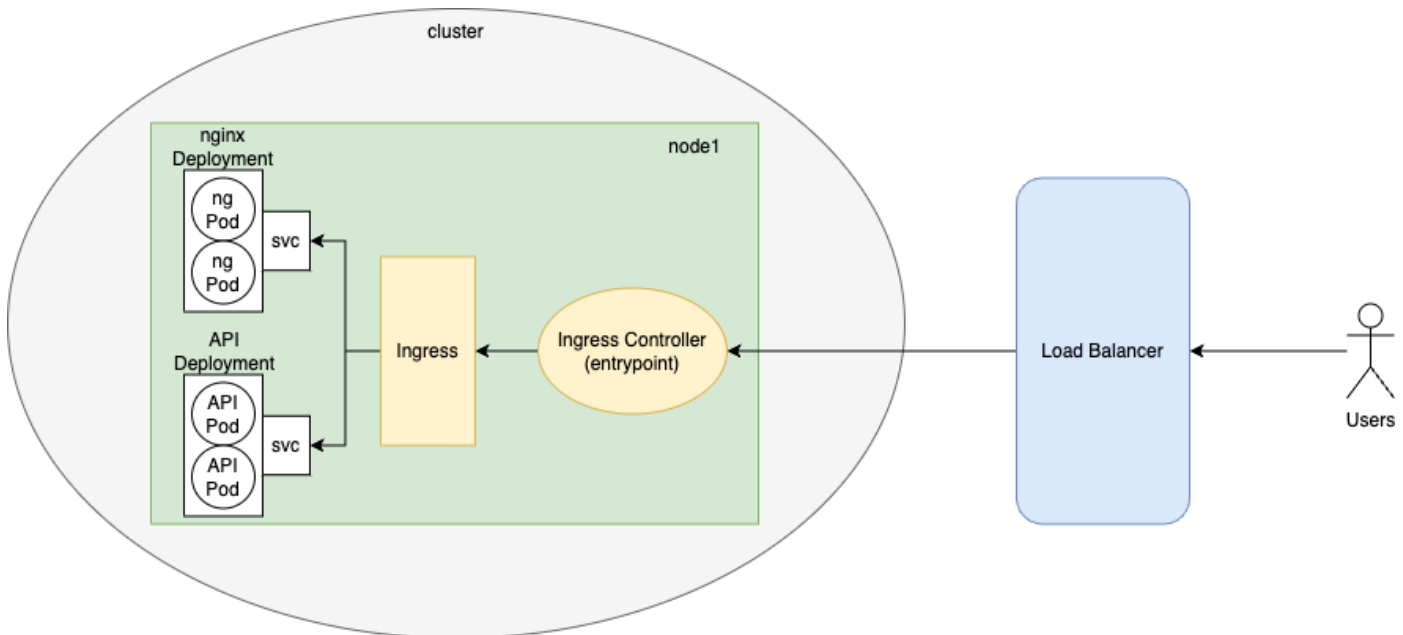
Basic concepts

These are the basic terms:

- **Node** is a server with Kubernetes installed on it.
- **Cluster** is a set of nodes. Usually one project runs on one cluster with multiple nodes.
- **Node pool** is a set of nodes with similar properties. Each cluster can have multiple node pools. For example, your cluster can have a "standard" pool and a "gpu" pool where in the gpu pool you have servers with powerful GPUs. Pools are mainly used for scalability.
- **Pod** is the smallest and simplest unit in Kubernetes. It's basically a container that runs on a given node. Technically, a pod can run multiple containers. We'll see some practical examples of that, but 99% of the time a pod == 1 container. In the sample application, each component (such API, frontend) will have a dedicated pod.
- **ReplicaSet** ensure that a specified number of replicas of a Pod are always running. In Docker Swarm it was just a configuration in the `docker-compose.yml` file. In Kubernetes it's a separate object.
- **Deployment**: everyone wants to run pods. And everyone wants to scale them. So Kubernetes provides us a deployment object. It merges together pods and replica sets. In practice, we're not going to write pods and replica sets but deployments. They define how applications are managed and scaled in the cluster. A deployment is a higher-level abstraction that manages Pods and provides features like scaling, rolling updates, and lifecycle management. It ensures that the desired number of Pods are always running and can be easily scaled up or down.
- **Services** provide network access to a set of Pods. They expose a given port of a set of pods and load balance the incoming traffic among them. Unfortunately, exposing a container (pod) to other containers is a bit more difficult than in docker-compose for example.
- **Ingress** and **ingress controller** are components that help us expose the application to the world and load balance the incoming traffic among the nodes.

- **Namespaces:** Kubernetes supports multiple virtual clusters within a physical cluster using namespaces. Namespaces provide isolation and allow different teams or applications to run independently without interference. Resources like Pods, Deployments, and Services can be grouped and managed within specific namespaces.

This is what it looks like:



This figure shows only one node, but of course, in reality we have many nodes in a cluster. Right now, you don't have to worry about the ingress components or the load balancer. The important thing is that there are deployments for each of our services, they control the pods, and there are services (svc on the image) to expose ports.

We'll have a load balancer (a dedicated server with the only responsibility of distributing traffic across the nodes), and the entry point of the cluster is this ingress controller thing. It forwards the request to a component called ingress, which acts like a reverse proxy and calls the appropriate service. Each service acts like a load balancer and they distribute the incoming requests among pods.

Pods can communicate with each other, just like containers in a docker-compose config. For example, nginx will forward the requests to the API pods (they run php-fpm).

I know it sounds crazy, so let's demystify it! First, a little bit of "theory" and then we start building stuff.

Pod

The smallest (and probably the most important) unit in Kubernetes is a Pod. It usually runs a single container and contains one component (such as the API) of your application. When we talk about autoscaling we think about pods. Pods can be scaled up and down based on some criteria.

A pod is an object. There are other objects such as services, deployments, replica sets, etc. In Kubernetes we don't have a single configuration file such as `docker-compose.yml` but many small(er) config files. Each object is defined in a separate file (usually, but they can be combined).

This is the configuration of a pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: api
spec:
  containers:
    - name: api
      image: martinjoo/posts-api:latest
      ports:
        - containerPort: 9000
```

As you can see, the `kind` attribute is set to `Pod`. Every configuration file has a `kind` key that defines the object we're about to create.

In `metadata` you can name your object with a value you like. It is going to be used in CLI commands, for example, if you list your pods, you're going to see `api` in the name column.

The `spec` key defines the pod itself. As I said, a pod can run multiple containers this is why the key is called `containers` not `container`. We can define the containers in an array. In this case, I want to run the `martinjoo/posts-api:latest` image and name the container `api`. `containerPort` is used to specify the port number on which the container listens for incoming traffic. We're going to talk about ports later. Right now, you don't have to fully understand it, it's just an example.

So this is a pod configuration. The smallest unit in k8s and it's not that complicated, actually.

The introduction continues in the book discussing other crucial k8s resources.

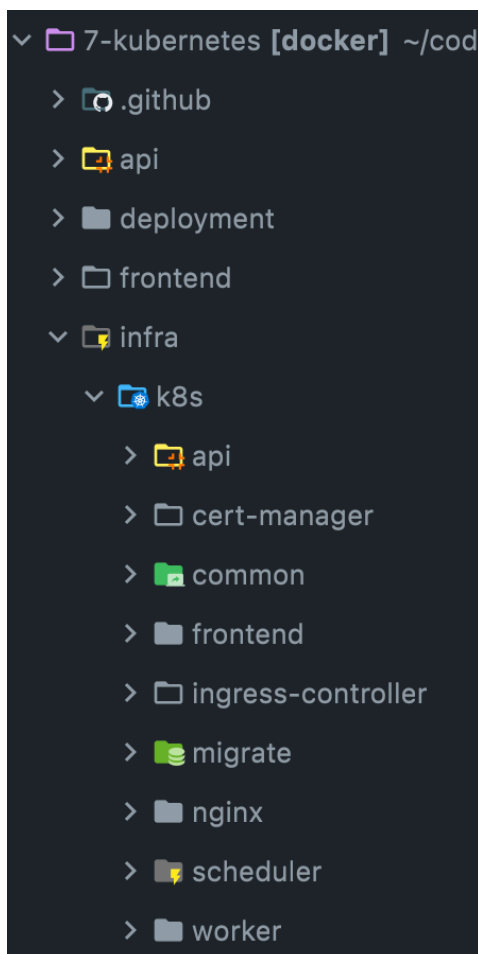
Deploying a Laravel API

Configuring the deployment

Let's start with the API. I'm going to create k8s related files inside the

```
/infra/k8s
```

folder. Each component (such as API or frontend) gets a separate directory so the project looks like this:



In the `api` folder create a file called `deployment.yml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 4
  selector:
```

```

matchLabels:
  app: api
template:
  metadata:
    labels:
      app: api
  spec:
    containers:
      - name: api
        image: martinjoo/posts-api:latest
        imagePullPolicy: Always
        ports:
          - containerPort: 9000

```

You've already seen a deployment similar to this one.

The image I'm using is `martinjoo/posts-api:latest`. In production, I never use the `latest` tag. It is considered a bad practice. It's a bad practice because you don't know exactly which version you're running and it's harder to rollback to a previous version if something goes wrong since the previous version was also `latest`... And also, `latest` if you're using some 3rd party images, `latest` is probably not the most stable version of the image. As the name suggests, it's the latest meaning it has the most bugs. Always use exact versions of Docker images. Later, I'm going to remove the `latest` tag and use commit SHAs as image tags.

The other thing that is new is the `imagePullPolicy`. It's a configuration setting that determines how a container image is pulled by k8s. It specifies the behavior for image retrieval when running or restarting a container within a pod. There are three values:

- `Always`: The container image is always pulled, even if it exists locally on the node. This ensures that the latest version of the image is used, but it means increased network and registry usage.
- `IfNotPresent`: The container image is only pulled if it is not already present on the node. If the image already exists locally, it will not be pulled again. This is the default behavior.
- `Never`: The container image is never pulled. It relies on the assumption that the image is already present locally on the node. If the image is not available, the container runtime will fail to start the container.

`IfNotPresent` is a pretty reasonable default value. However, if you use a tag such as `latest` or `stable` you need to use `Always`.

A note about `containerPort`. As I said in the introduction, exposing ports in k8s is a bit more tricky than defining two values in a YAML file. This `containerPort` config basically **does nothing**. It won't expose port 9000 to the outside world or not even to other containers in the cluster. It's just an information that is useful for developers. Nothing more. Later, we're going to expose ports and make communication possible between components.

The chapter continues in the book discussing everything related to deploying the API. After that we create resource and deploy all the other parts of the app such Vue frontend, workers, scheduler, etc.

This is the sample chapter of DevOps with Laravel. You can find the book [here](#).